Zara Nails Project Audit and Multi-Tenant Strategy

Executive Summary

This project is already a feature-rich single-salon platform with:

- Client app (booking, appointments, profile, loyalty, messaging, waiting list, sharings)
- Admin app (dashboard, clients, bookings, scheduling/calendar, messaging, loyalty management, reports, birthdays, gallery)
- Firebase backend + Cloud Functions for automation (emails, push, reminders, birthday campaigns, secure booking/reschedule validation)
- Cross-platform delivery (React Native + Expo for iOS/Android/Web)

For your broader SaaS direction (multi-tenant, multi-location, multi-technician, tenant branding, engagement modules), the best path is:

- Start a fresh product codebase for core architecture and data model
- Selectively port proven business logic and UI patterns from this repo
- Keep this repo as a stable reference and source of reusable modules

This gives you scalable architecture from day one while preserving your existing investment.

---

1) Current Implemented Features (Feature Inventory)

A. Platform and Architecture

- Cross-platform app (iOS/Android/Web) using React Native + Expo.
- Real-time Firestore listeners for users, services, bookings, messages, waiting list, gallery.
- Role model implemented: client, admin, owner.
- Environment strategy for dev/prod Firebase projects and deploy scripts.

Evidence:

- App shell and routing: `App.js`
- App state and data layer: `src/context/AppContext.js`
- Build/deploy scripts: `package.json`
- Firebase setup docs: `Documentation/03-Developer-Guides/DEVELOPER_HANDOVER_DOCUMENTATION.md`

B. Authentication and User Lifecycle

- Email/password registration and login.
- Admin approval flow for new client registrations.
- Password reset.
- Client creation by admin.
- Role promotion/demotion (client <-> admin, owner controls).
- User activation/deactivation and delete.

Evidence:

- Auth APIs and role/user ops: `src/context/AppContext.js`
- Login UI: `src/screens/LoginScreen.js`
- Register UI + referral input + consent flags: `src/screens/RegisterScreen.js`

C. Profile, Privacy, and Account Controls

- Client profile editing (name, phone, instagram, DOB).
- Notification preference management.
- Email change flow (self-service verification + admin-managed forced flow).
- GDPR-like features: export user data and delete account.
- Privacy policy / terms visibility from profile.

Evidence:

- Profile UI and actions: `src/screens/ProfileScreen.js`
- Email sync/change logic: `src/context/AppContext.js`
- Admin email-change cloud function: `functions/index.js` (`adminUpdateClientEmail`)

D. Service Catalog and Pricing

- Admin can add/update/deactivate services.
- Service sorting/reordering support via sortOrder.
- Currency setting support (owner-level from dashboard).

Evidence:

- Service CRUD and reorder: `src/context/AppContext.js`
- Admin services screen: `src/screens/admin/AdminServicesScreen.js`
- Currency selector in admin dashboard: `src/screens/admin/AdminDashboardScreen.js`

E. Booking and Appointment Lifecycle

- Service/date/time booking by client.
- Slot availability checks against existing bookings.
- Server-side secure booking creation to prevent race-condition double booking.
- Booking statuses: pending, confirmed, completed, rejected, cancelled, reschedule_pending, reschedule_rejected, rescheduled.
- Client cancellation and reschedule request flow.
- Admin confirms/rejects bookings and reschedule requests.
- Admin direct reschedule of bookings.
- Admin swap appointments between two clients.
- Appointment image attachments (reference photos).

Evidence:

- Booking logic: `src/context/AppContext.js`
- Client booking UI: `src/screens/BookingScreen.js`
- Client appointments UI and reschedule modal: `src/screens/AppointmentsScreen.js`

- Admin booking management: `src/screens/admin/AdminBookingsScreen.js`
- Secure booking/reschedule endpoints: `functions/index.js` (`createBookingSecure`, `approveRescheduleSecure`)

F. Availability and Schedule Management

- Admin day-level slot management (enable/disable slots, custom slot creation).
- Calendar and schedule views with booking density indicators.
- Action controls per occupied slot (complete, swap, reschedule, cancel).

Evidence:

- Availability APIs: `src/context/AppContext.js` (`setDayAvailability`, `getAvailableSlots`)
- Schedule UI: `src/screens/admin/AdminScheduleScreen.js`
- Calendar UI: `src/screens/admin/AdminCalendarScreen.js`

G. Waiting List System

- Client joins waiting list by date range.
- Admin sees and manages waiting list entries.
- Automatic notifications when slots open due to cancellation/reschedule.
- Auto-removal from waiting list after successful booking in target window.

Evidence:

- Waiting list API and listeners: `src/context/AppContext.js`
- Waiting list client UI: `src/screens/WaitingListScreen.js`
- Waiting list docs: `Documentation/05-Features-Implementation/WAITING_LIST_FEATURE.md`
- Waiting list notification pipeline: `functions/index.js` (`notifyWaitingList`, `onMessageCreated` branch)

H. Messaging and Communication

- In-app messaging between clients and admin/owner.
- Read/unread tracking.
- Attachments in chat (images/files).
- Bulk messaging to selected client segments.
- Email + push notifications triggered from message events.

Evidence:

- Messaging APIs: `src/context/AppContext.js` (`sendMessage`, `sendBulkMessage`)
- Client messages: `src/screens/MessagesScreen.js`
- Admin inbox and bulk messaging UI: `src/screens/admin/AdminMessagesScreen.js`
- Message-triggered email/push backend: `functions/index.js` (`onMessageCreated`)

I. Push and Email Notification Automations

- Push token registration and persistence.
- Booking-status notifications (email + push).

- Daily appointment reminder automation.
- Daily birthday email automation.
- Manual callable triggers for reminders and birthday sends.

Evidence:

- Push token registration in app nav lifecycle: `App.js`
- Notification utility: `src/utils/notifications.js`
- Cloud Functions automations: `functions/index.js` (`sendDailyAppointmentReminders`, `sendDailyBirthdayEmails`, `onBookingUpdated`)
- Notification docs: `Documentation/05-Features-Implementation/NOTIFICATIONS.md`

J. Loyalty and Referral Program

- Loyalty levels/tiers with configurable rules.
- Loyalty points accrual for completed appointments and cadence bonus.
- Referral codes and referral reward processing.
- Social share reward with monthly limits.
- Loyalty transaction ledger per user.
- Client-facing loyalty screen + badge.
- Admin loyalty overview + per-client manual adjustments and redemptions.

Evidence:

- Loyalty engine and transactions: `src/context/AppContext.js`
- Client loyalty UI: `src/screens/ClientLoyaltyScreen.js`
- Badge component: `src/components/LoyaltyBadge.js`
- Admin loyalty summary: `src/screens/admin/AdminLoyaltyProgramScreen.js`
- Loyalty docs: `Documentation/05-Features-Implementation/loyalty-implementation.md`

K. Gallery and Social Content

- Admin-managed gallery for showcasing work.
- Client sharings upload flow.
- Admin sharings moderation/mark-viewed workflow.

Evidence:

- Gallery/sharing APIs: `src/context/AppContext.js`
- Gallery screen: `src/screens/InstagramGalleryScreen.js`
- Share screen: `src/screens/ShareNailsScreen.js`
- Admin gallery/sharings: `src/screens/admin/AdminGalleryScreen.js`, `src/screens/admin/AdminSharingsScreen.js`

L. Reporting and Retention Analytics

- Retention KPI metrics: retention rate, rebooking rate, avg days between, at-risk count.
- Service-level loyalty metrics.
- At-risk client classification with urgency levels.
- Never-booked client identification.

- Action links from report cards into client management.

Evidence:

- Reports implementation: `src/screens/admin/AdminReportsScreen.js`
- Retention methodology docs: `Documentation/05-Features-Implementation/RETENTION_REPORTS_DOCUMENTATION.md`

M. Birthdays and Engagement

- Birthday-oriented client list views (today, window, month filters).
- Admin birthdays screen linked to client profile.
- Automated birthday emails via Cloud Functions.

Evidence:

- Birthday UI: `src/screens/admin/AdminBirthdaysScreen.js`
- Birthday automation: `functions/index.js` (`sendDailyBirthdayEmails`)

N. Localization

- Multi-language translation system using context + AsyncStorage persistence.
- Large translation dictionary currently in place.

Evidence:

- Language context: `src/context/LanguageContext.js`
- Translation catalog: `src/localization/translations.js`

O. Testing and Documentation

- Jest setup and test suites for core contexts/screens.
- Significant internal documentation: architecture, setup, feature implementation, migration/multitenancy.

Evidence:

- Tests: `__tests__/`
- Docs root: `Documentation/`

---

2) Existing Features to Carry Into Your Multi-Tenant Product

Below is the practical carry-forward set for your broader platform.

Keep and Port (High Reuse Value)

- Auth foundations and role-based access flow.
- Booking lifecycle logic and secure server-side slot validation.
- Reschedule and swap operational workflows.
- Waiting list mechanics.

- Messaging model + bulk communication patterns.
- Notification automation patterns (event triggers + scheduled jobs).
- Loyalty ledger model and transaction history structure.
- Retention KPI framework and risk segmentation model.
- Multilingual infrastructure.

Keep but Rework for Multi-Tenant

- Service catalog: from global salon catalog to per-tenant and per-location catalogs.
- Availability model: from salon-level day slots to technician-level shifts/blocks and location timezone-aware scheduling.
- Gallery/sharing: scope by tenant/location and moderation policies.
- Reports: tenant-scoped, location-scoped, technician-scoped segmentation.
- Birthday and campaign flows: tenant branding + localization + legal consent model.

Add New (Not Present or Only Partially Present)

- First-class tenant model (`tenantId`) with strict data isolation.
- Multi-location structure per tenant.
- Employee/technician roster and workload balancing across locations.
- Brand/theming per tenant (colors/logo/voice) at runtime.
- Reviews and ratings model with moderation and anti-abuse controls.
- Marketing engine (segments, campaigns, templates, automations).
- Daily/weekly activities/challenges and gamified engagement loops.
- Marketplace discovery feed, salon public profiles, and social-style posts with "Book this look".
- Salon onboarding wizard (account, profile, services, staff, policies, availability, launch verification).
- Client onboarding aligned with existing flow (guest path + full account upgrade path).
- Free trial lifecycle (activation triggers, countdown, expiry behavior, reactivation flow).
- **Multi-Salon User Access Model**: Single user account can subscribe to multiple salons; unified home dashboard with salon context switching and per-salon unread message badges.
- **Subscription and Billing**: Tiered subscription model (monthly/annual billing), Stripe integration, feature gating per subscription level, 14-day free trial, payment failure handling with grace period and suspension indicator.
- AI feature set: scheduling optimization, marketing automation, retention predictions, content generation, chat assistance, no-show/fraud prediction, marketplace personalization.
- Tenant billing/subscriptions, onboarding wizard, and admin portal.
- Fine-grained RBAC (platform admin, tenant owner, location manager, technician).

---

3) High-Level Multi-Tenant Project Plan (Expanded)

Phase 0: Product and Domain Foundation

- Define domain entities: Tenant, Location, Staff, Service, Customer, Booking, Campaign, Review, Challenge.
- Define access model and permission matrix for platform roles and tenant roles.
- Define non-functional targets: SLA, scaling limits, observability, compliance.

Phase 1: Core Multi-Tenant Backbone

- New repository and clean architecture.
- Core data model with mandatory tenant scoping.
- Tenant-aware auth/session middleware.
- Tenant-scoped Firestore rules and indexes (or alternative backend if chosen).

Phase 2: Multi-Location and Multi-Technician Scheduling

- Build location and technician management modules.
- Technician availability and exceptions (breaks, leave, capacity).
- Booking assignment engine with conflict prevention and fallback logic.
- Timezone and locale-safe scheduling.

Phase 3: White-Label and Customization

- Tenant theme tokens (color, typography, iconography, imagery).
- Runtime theme loading in mobile/web shells.
- Tenant custom settings (booking windows, cancellation policy, loyalty rules).

Phase 4: Migration of Existing High-Value Features

- Port booking lifecycle and secure validation flows.
- Port messaging and notification automation patterns.
- Port loyalty and retention analytics with tenant scoping.
- Port waiting list and birthday automation.

Phase 5: New Engagement and Growth Layer

- Reviews and ratings module.
- Marketing capabilities: campaign builder, audience segments, trigger automation.
- Daily/weekly activity feeds and challenge mechanics.
- Social proof and referral funnels.

Phase 6: Platform Operations and Monetization

- Tenant onboarding wizard and sample data seeding.
- **Subscription Management and Billing**:
    - Multi-tier subscription model (Starter, Professional, Enterprise)
    - Monthly and annual billing (annual with discount)
    - Stripe payment processing and webhook handling
    - 14-day free trial for new tenants
    - Feature gating based on subscription tier
    - Payment failure handling with grace period and suspension indicator
    - Invoice and receipt generation
- **Multi-Salon User Access** (for end-users):
    - User home dashboard listing all subscribed salons
    - Unread message badge aggregation per salon
    - Quick context switching between salons

    - Next appointment preview per salon
    - Loyalty points balance visibility per salon
- Audit logs, admin console, incident tooling.
- Analytics warehouse/export and business intelligence views.

Phase 7: Launch and Scale

- Pilot with Zara as Tenant 1 in the new platform.
- Add 2-3 additional pilot salons.
- Load test and optimize hot paths.
- Rollout playbook and migration guide for future tenants.

Phase 8: Marketplace and Onboarding Expansion

- Full marketplace launch (feed, profiles, search filters, post interactions).
- Anti-client-theft guarantees in booking flow (no competitor injection during booking).
- Salon onboarding wizard v1 with completion score and resumable steps.
- Client onboarding v1 merged into booking and discovery touchpoints.

Phase 9: AI Differentiation Layer

- AI chat assistance rollout for salon/admin/client contexts.
- AI scheduling and no-show prediction models with explainability constraints.
- AI marketing and content generation with consent and policy safeguards.
- AI marketplace personalization using behavior and preference signals.

Delivery Horizon Update

- The original 12-week plan is no longer sufficient for approved scope.
- Recommended delivery horizon: **20 weeks**.
- Weeks 1-12 remain core foundation and pilot readiness.
- Weeks 13-20 cover payment hardening, onboarding expansion, marketplace, and AI delivery.

---

4) Fork/Branch Existing Project vs Start Fresh

Current Codebase Reality (Important)

- The current app is functionally rich but architecturally centered on a single salon model.
- Core collections and rules are globally scoped (no enforced tenant/location partitioning).
- The app state layer is monolithic and would require broad, high-risk refactoring for true tenancy.
- There is documented legacy drift in settings keys and some temporary operational utilities.

Option Analysis

Option A: Branch/Fork Existing Repo

Pros:

- Faster short-term feature velocity.

- Immediate reuse of many UI screens and workflows.

Cons:

- High refactor risk to retrofit strict tenant boundaries everywhere.
- Easier to introduce data-leak bugs between tenants.
- Harder long-term maintainability (legacy + new concerns intertwined).
- Increased QA burden due to architectural mutation.

Option B: Fresh New Repo (Recommended)

Pros:

- Multi-tenant architecture first, not retrofit.
- Cleaner domain boundaries and long-term maintainability.
- Better security posture and easier isolation guarantees.
- Cleaner onboarding for future developers and teams.

Cons:

- Slightly longer initial build phase.
- Requires deliberate migration/porting plan.

Recommendation

Choose fresh new repo + selective code transplant.

Execution model:

1. Build new multi-tenant core and data model.
2. Port proven logic modules from this repo in order of value:
    - Secure booking validation
    - Booking lifecycle transitions
    - Waiting list
    - Messaging + notifications
    - Loyalty + retention analytics
3. Rebuild UI screens on top of tenant-aware services.
4. Use Zara as first production tenant in new platform.

This aligns with the project's own migration documentation and avoids expensive architecture debt.

Strategic decision update (April 2026):

- Include a public landing entry as part of core architecture, not as a later add-on.
- Use route split from day one: Public (Landing/Login/Register/Discover) vs Protected (authenticated app).
- Add a marketplace-ready discovery scaffold early, backed by mocked data initially, so future marketplace expansion does not require navigation/domain refactor.
- Keep support, admin, and tenant data operations strictly in protected routes.

---

5) Design Tooling Recommendation (External AI vs Copilot)

Short Answer

Use both:

- External design tool for high-fidelity visual system and brand exploration
- Copilot (this workflow) for implementation-grade architecture and production code

Recommended Stack for Your Use Case

- Primary design: Figma (with Figma AI + design tokens workflow)
- Rapid concept generation: Galileo AI or Uizard for quick mobile concepts
- Web interaction prototyping: Framer or v0-style prototype generators (for marketing/landing and flow validation)
- Design-to-code handoff: Tokens Studio + strict component specs (avoid raw auto-generated code in production)

Why This Split Works

- External AI design tools are better at visual exploration, moodboards, and quick variant generation.
- Copilot is better for production logic, state management, API integration, and platform correctness.
- For mobile-first web + native apps, consistency requires a design system, not one-off generated screens.

Practical Workflow

1. Define 2-3 visual directions in Figma (each with full mobile-first flows).
2. Lock one design language and create tokenized design system.
3. Implement shared components in code with strict token usage.
4. Port flows incrementally (auth, booking, messaging, loyalty, engagement).
5. Measure user behavior, then iterate visuals and conversion paths.

---

6) Risks and Mitigations for the New Program

Key Risks

- Data isolation bugs in early multi-tenant rollout.
- Over-scoping (reviews + campaigns + challenges all at once).
- Migration delays if feature parity is not prioritized.

Mitigations

- Enforce tenantId in every write path and security rule from day zero.
- Deliver in phases: parity first, then growth features.
- Build tenant-aware test fixtures and automated integration tests early.
- Run Zara as pilot tenant before scaling tenant onboarding.

---

7) Suggested Next Actions

1. Approve fresh-repo strategy and target architecture.
2. Lock MVP scope for parity (features to carry in first 8-12 weeks).
3. Define multitenant Firestore schema v1 and role matrix.
4. Create UI system brief for Figma and generate first 2 visual concepts.
5. Execute landing/discovery foundation tasks early:
    - Week 1: Public Landing Shell and route groups.
    - Week 2: Discover directory scaffold (marketplace-ready entry).
6. Start implementation phases with Zara as Tenant 1 pilot.

Appendix: Key Source References Used in This Audit

- `App.js`
- `src/context/AppContext.js`
- `src/context/LanguageContext.js`
- `src/screens/*.js`
- `src/screens/admin/*.js`
- `functions/index.js`
- `firestore.rules`
- `package.json`
- `Documentation/03-Developer-Guides/DEVELOPER_HANDOVER_DOCUMENTATION.md`
- `Documentation/05-Features-Implementation/*.md`
- `Documentation/08-Migration-MultiTenant/*.md`